



Санкт-Петербургский государственный университет
Кафедра системного программирования

Сравнение производительности алгоритмов на графах из SPLA/GraphBLAS+LAGraph

Гавриленко Михаил, группа 23.Б15-мм

Санкт-Петербург
2023

Постановка задачи

Целью является исследование алгоритмов BFS, SSSP, TC, PR из библиотек SPLA и GraphBLAS/LAGraph

Задачи:

- Сформулировать набор гипотез о производительности алгоритмов на основе базовых представлений о наивном алгоритме и особенностях платформ
- Произвести замеры производительности реализаций из выбранных библиотек
- Исследовать результаты и выявить особенности, отразившиеся на производительности
- Сделать выводы о ранее сформулированных гипотезах

Использованные метрики:

- Время выполнения (мс)
- Степень прироста производительности SPLA относительно LAGraph
- GTEPS(Giga Traversed Edges Per Second)

Конфигурация системы:

- M2 Max (arm64), 38-гпу ядер 1000МГц, 12-спу ядер (8+4), 32ГБ ОЗУ
- OpenCL 1.2
- clang 17.0.0

BFS: (первичные тезисы)

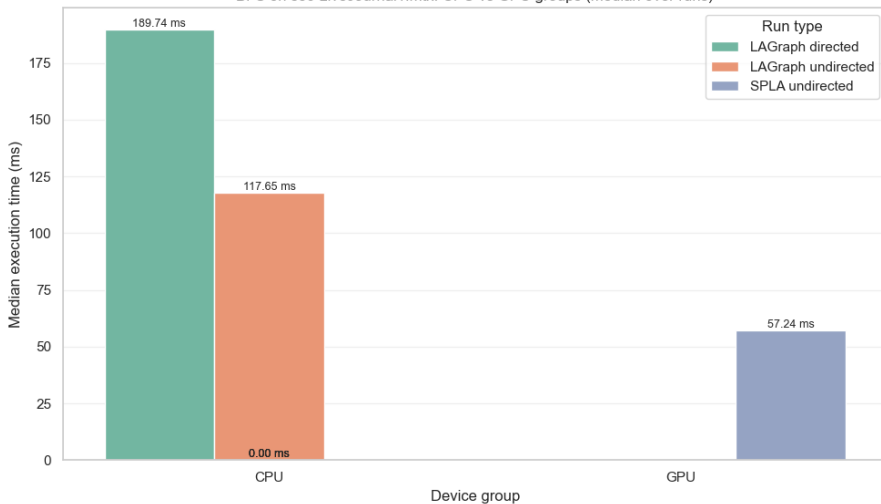
- Исполнение алгоритма эффективнее на GPU
- GPU лучше справляется с симметричными датасетами, в отличие от CPU
- Преобразование в неориентированный граф увеличит вычислительную нагрузку, но улучшит показатель GTEPS за счет более эффективной работы Pull-фазы (bottom-up BFS)

BFS: (детали алгоритмов)

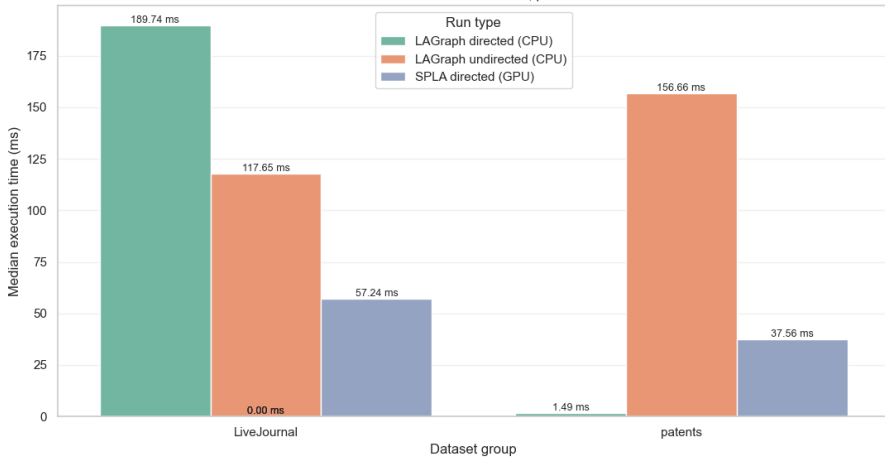
Оба алгоритма являются "pull-push", однако некоторые детали различаются:

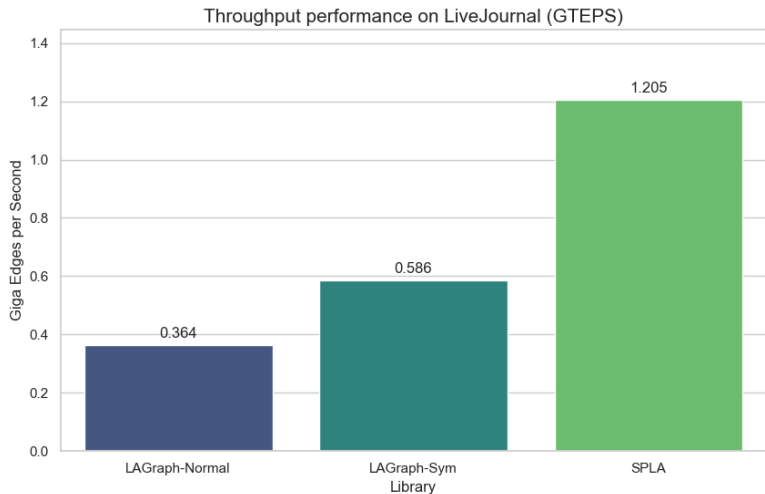
- SPLA выбирает push/pull по плотности текущей фронтальной структуры
- LAGraph опирается на кэшированные свойства, в ином случае переключается на push-only
- В SPLA есть явный early-exit

BFS on soc-LiveJournal1.mtx: CPU vs GPU groups (median over runs)



Median BFS time: LiveJournal first, patents second

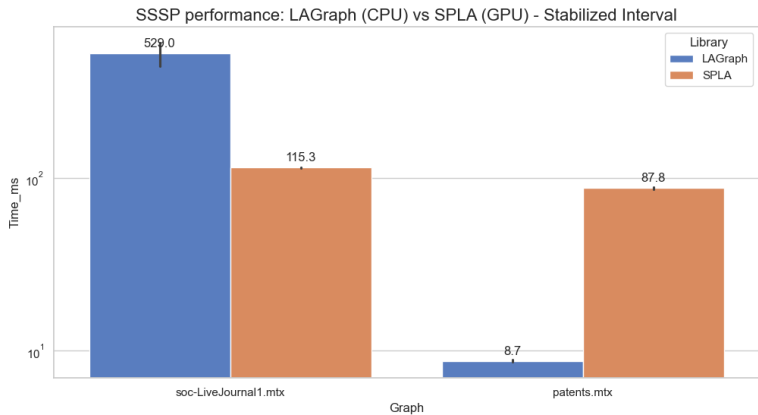


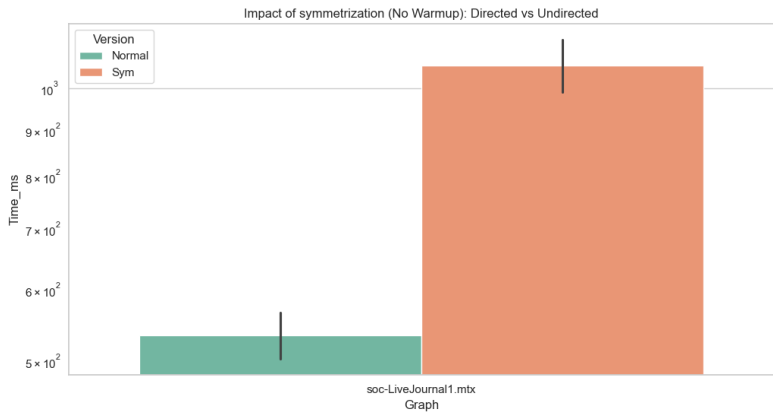


Выводы:

- Выполнение большого кол-ва итераций с маленьким фронтиром - отрицательно действует на производительность GPU
- Время обхода направленного графа зависит от его свойств - "длины" путей и "ширины" фронтиров. Высокий коэффициент кластеризации и малый диаметр соцсети позволяют быстро расширять фронт и эффективно использовать параллелизм
- SPLA эффективнее на больших графах с большой компонентой связности. На малых графах или при высокой фрагментации LAGraph предпочтительнее из-за низкой задержки CPU

- Исполнение алгоритма эффективнее на GPU
- GPU лучше справляется с симметричными датасетами, в отличии от CPU GPU (SPLA) покажет значительное преимущество на графах с плотной структурой и широким фронтиром
- Преобразование ориентированного графа в неориентированный (Sym) радикально увеличит время выполнения
- Первая итерация на GPU будет значительно медленнее последующих из-за JIT-компиляции ядер OpenCL

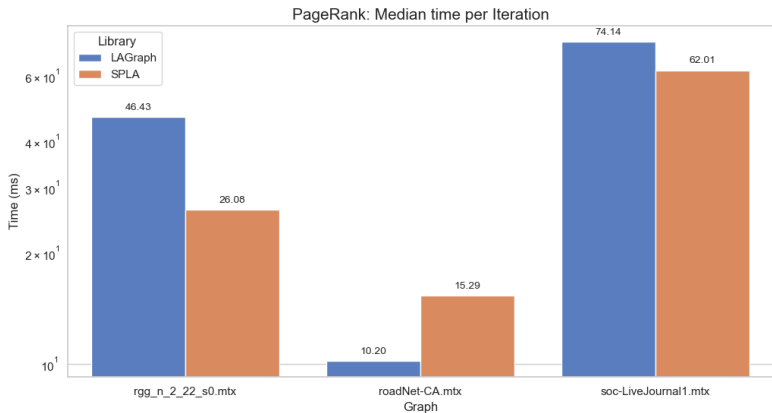




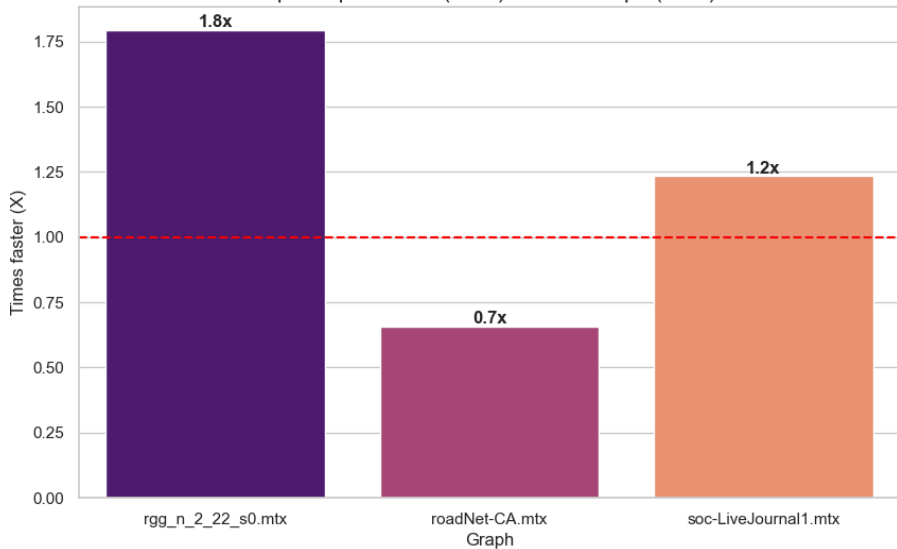
- В графе патентов (где 3.7 млн узлов) вершина с наибольшим кол-вом исходящих ребер связана только с 776 другими вершинами. У LiveJournal вершина имеет в 20 раз больше выходящих ребер
- На LiveJournal GPU работает в 4.7 раза быстрее CPU. Это "родная" среда для массивного параллелизма SPLA
- Гипотеза об оверхеде при симметрии подтверждена полностью

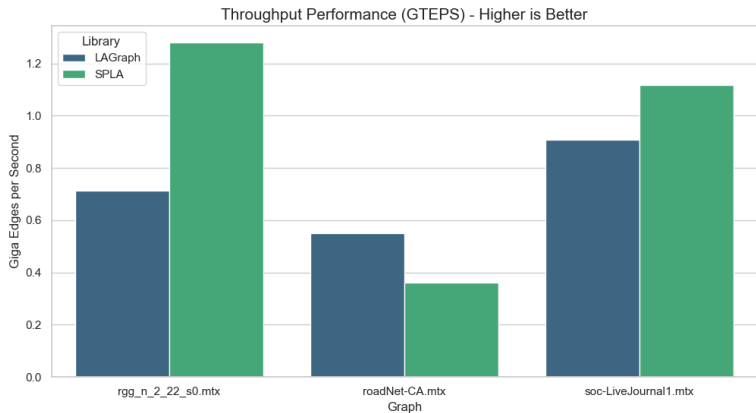
Общий вывод: Для алгоритма SSSP выбор вычислительного устройства критически зависит от топологии.

- Эффективность GPU-ускорения прямо пропорциональна регулярности структуры графа
- Низкая средняя степень узла делает использование GPU нецелесообразным
- Неравномерность распределения в безмасштабных сетях снижает потенциал GPU по сравнению с регулярными структурами



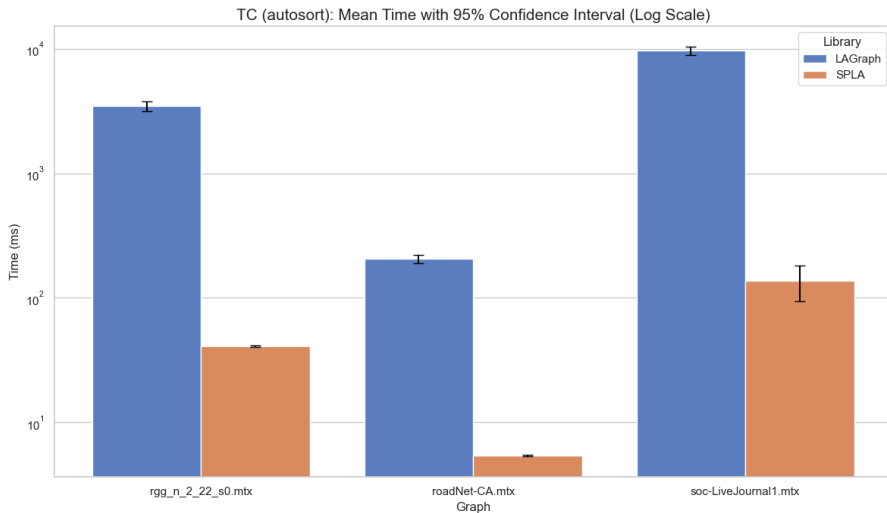
Speedup of SPLA (GPU) over LAGraph (CPU)



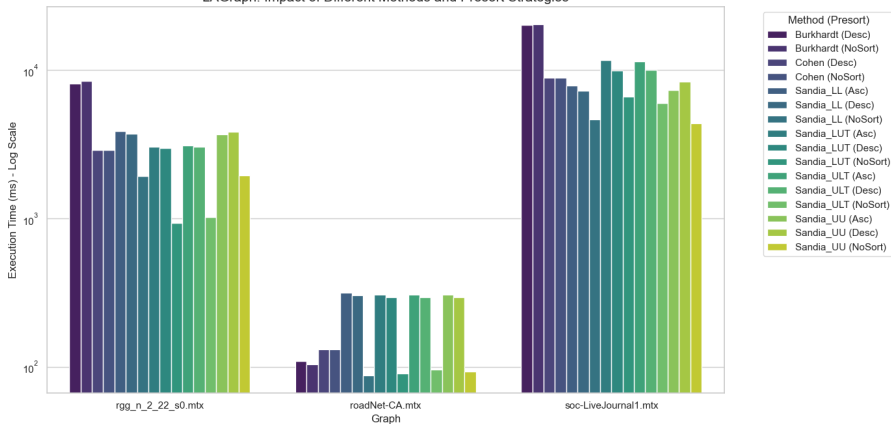


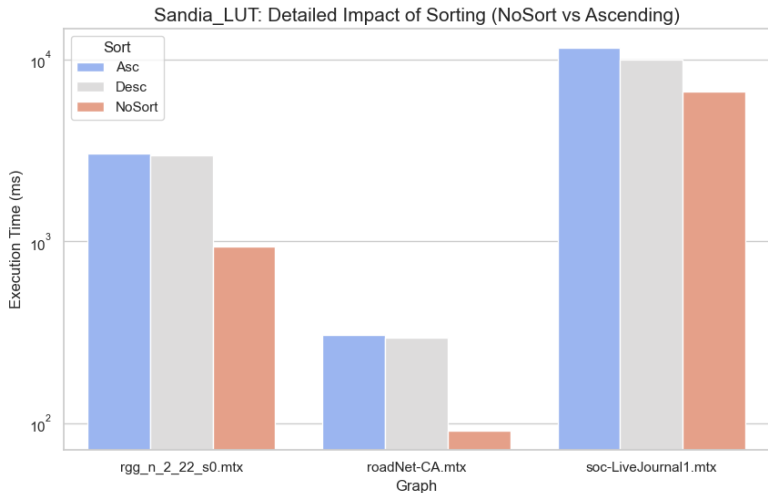
- Степени узлов распределены равномерно, что минимизирует простой ядер GPU
- Дорожные графы эффективнее на CPU. Накладные расходы на запуск ядер и обращение к памяти GPU не окупаются малым объемом вычислений на одно ребро. **Общий вывод:** SPLA эффективен на "тяжелых" графах с большим количеством ребер, но его преимущество нивелируется при сильной разреженности (дороги) или экстремальной неравномерности (социальные сети).

- Архитектура GPU должна показывать многократное преимущество в данной задаче
- На разреженных графах с низкой средней степенью/ (road) эффективность GPU будет ниже, чем на плотных или регулярных из-за накладных расходов
- Для алгоритмов семейства Sandia на CPU предварительная сортировка узлов по степени критически важна для минимизации количества вычислений



LAGraph: Impact of Different Methods and Presort Strategies





- Ускорение на GPU достигает 84.5x. Задача Triangle Counting, основанная на пересечении множеств, идеально ложится на архитектуру SPLA
- Высокий GTEPS на дорогах говорит о том, что маскирование в SPLA работает исключительно эффективно даже при малом количестве треугольников
- Предположение о том, что Ascending Sort всегда ускоряет TC на CPU, опровергнуто. Накладные расходы на манипуляцию данными в памяти делают сортировку неэффективной для крупных графов
- Самой быстрой стратегией для TC на CPU является использование наиболее эффективного алгоритма (Sandia LUT или LL) на исходном порядке узлов (NoSort), чтобы избежать дорогостоящих операций перестановки матриц